

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA5112

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO (BL4, BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

CASE STUDY

Code of Conduct:

I G. Srivalli (192365014) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Srivalli

Annexure A

1. Weak Password Hashing in Web Application

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice.

2. Digital Signature Compromise in Financial System

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.

Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

3. Kerberos Authentication Failure in Distributed System

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

4. Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

5. Integrity Verification in Cloud Data Storage

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.

Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Weak Password Hashing in Web Application

Case: A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables. The task is to analyze the effect of missing salts and SHA-1 and recommend a secure password hashing strategy.

Introduction

Password storage should not use a fast general-purpose hash as if it were a password-protection mechanism. A secure password-storage design uses a password hashing or key-derivation function that is deliberately expensive, applies a unique random salt to every password, and stores the algorithm parameters needed for later verification. OWASP recommends modern adaptive password hashing such as Argon2id, with scrypt, bcrypt, or PBKDF2 as alternatives when appropriate.

Why the Missing Salt Is Dangerous

- With no salt, identical passwords produce identical SHA-1 values. An attacker can therefore recognize reused passwords across accounts without cracking every account separately.
- Precomputed dictionaries or rainbow-table style lookups can be prepared in advance because the same password always maps to the same digest.
- Once one common password is identified, every account using that password has the same stored hash and becomes an immediate target.
- Salting makes each password instance unique. A different random salt forces an attacker to recompute the password guess for each account instead of reusing one precomputed result. OWASP specifically identifies salting as a defense against precomputation and large-scale hash reuse.

Why SHA-1 Is Inappropriate for Password Storage

SHA-1 is designed as a fast cryptographic hash, not as a password hashing function. Password crackers benefit from the same speed that helps legitimate applications compute SHA-1 quickly: an attacker can test very large numbers of guesses. OWASP therefore recommends dedicated password hashing functions that are intentionally slow and configurable, rather than fast functions such as SHA-1.

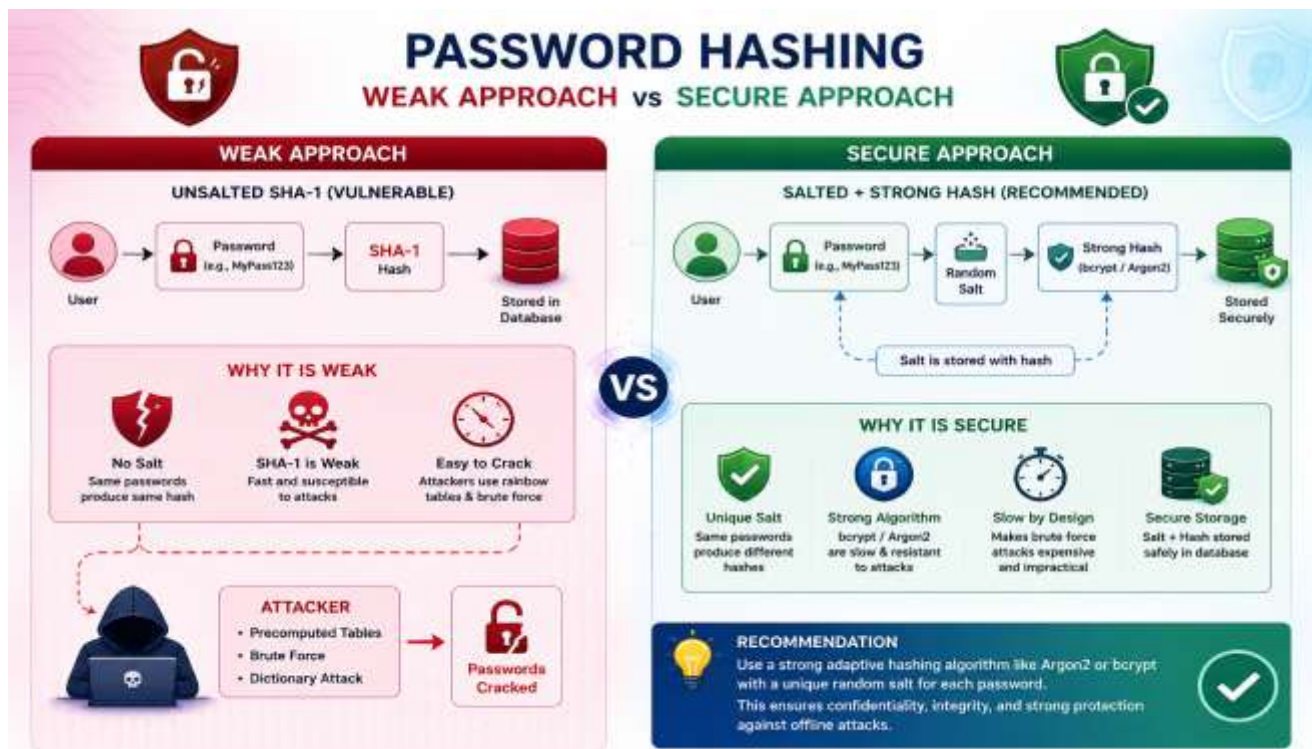


Figure 1: Secure password storage flow using a unique salt and Argon2id

Recommended Secure Strategy

The preferred modern strategy for a new application is Argon2id with a unique, cryptographically random salt for every password. The application should use a tuned memory, time, and parallelism cost so verification is practical for legitimate users but substantially more expensive for attackers. OWASP describes Argon2id as the preferred choice among the password-hashing options it discusses and recommends tuning its memory, iteration, and parallelism parameters to the deployment environment.

- Generate a new random salt for every password and store it with the password-hash record; the salt does not need to be secret.
- Hash the password with Argon2id using a cost configuration selected through benchmarking on the production-class server.
- Store the algorithm identifier and parameters with the resulting hash so the application can verify and later upgrade the work factor.
- On successful login, rehash old records using the current Argon2id policy when the stored parameters are weaker.
- Add rate limiting, MFA for sensitive accounts, secure password-reset procedures, and breach monitoring as complementary controls.

Analysis and Justification

The central weakness is not simply that SHA-1 is an old algorithm; it is that the application combines a fast hash with no per-user salt. This allows attackers to scale password guessing efficiently and to reuse work across many accounts. Argon2id directly addresses the economics of password cracking by increasing computational and memory cost while salting prevents the same precomputed result from being reused across accounts. Therefore, Argon2id with unique salts provides a substantially stronger password-storage design than unsalted SHA-1.

Conclusion

The breach became much more damaging because unsalted SHA-1 allowed password guesses to be reused and computed cheaply. Replacing the legacy scheme with a properly configured, salted, adaptive password hash such as Argon2id is the recommended core remediation.

2. Digital Signature Compromise in Financial System

Case: A financial institution uses digital signatures for transaction approval. An attacker obtains a user's private key and performs unauthorized transactions. The task is to evaluate the effect on authentication and non-repudiation and propose mitigation strategies.

Introduction

A digital signature is created using a private key and verified with the corresponding public key. Properly implemented digital signatures provide origin authentication, integrity protection, and support non-repudiation. NIST describes digital signatures as a mechanism that can detect unauthorized modification, authenticate the signatory, and provide evidence that a signature was generated by the claimed signatory.

Impact of Private-Key Compromise

- Authentication is undermined because anyone possessing the private key can generate signatures that verify under the user's public key. The system can no longer distinguish the legitimate user from the attacker solely through signature verification.
- Transaction integrity is also affected because the attacker can create valid-looking signatures over unauthorized transaction data.
- Non-repudiation is weakened in practice. A verifier sees a mathematically valid signature associated with the user's public key, but the key was no longer under exclusive control of the legitimate signer. NIST's definition of non-repudiation relies on assurance that the relevant private key is associated with the signatory.
- The incident therefore becomes a key-compromise event, not merely a stolen password event. All transactions signed with the compromised key may need investigation and possible repudiation procedures.

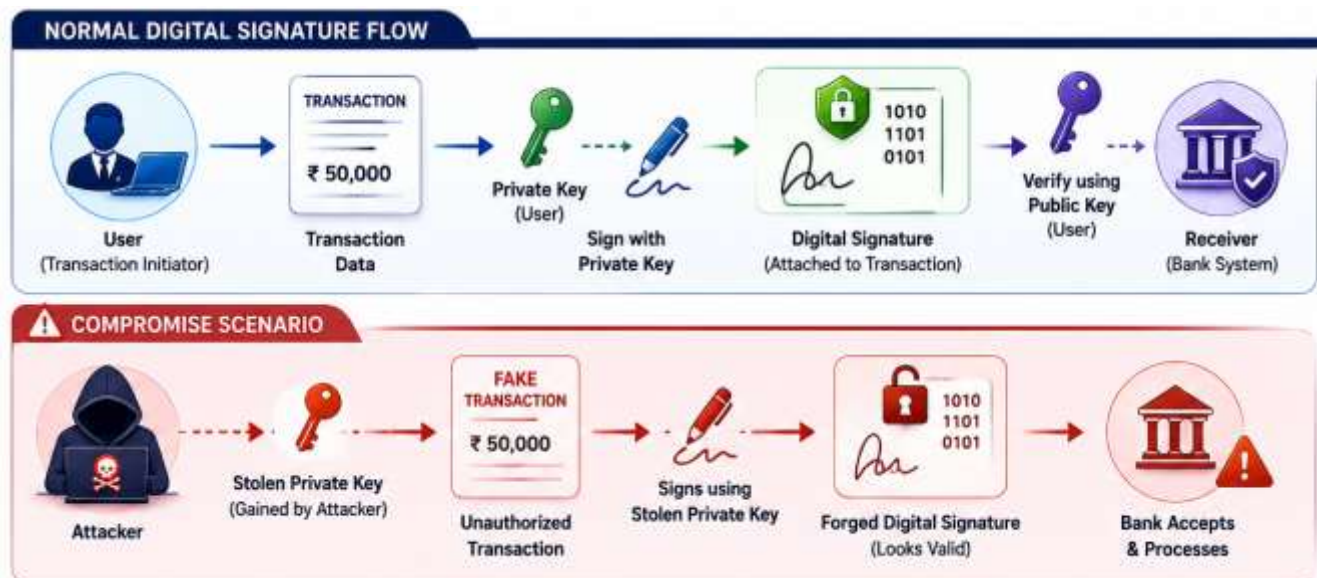


Figure 2: Digital-signature workflow with protected private-key storage

Mitigation Strategies

Control	Purpose
Hardware-backed key protection / HSM	Keeps private-key operations inside a hardened device and reduces exposure of raw private-key material.
Strong access control + MFA	Adds an independent authentication layer before a signing operation can be requested.
Transaction signing / step-up approval	Requires explicit confirmation of high-value or high-risk transactions rather than allowing silent signing.
Key rotation and immediate revocation	Limits the lifetime of a compromised key and prevents continued acceptance after compromise is detected.
Audit logs and anomaly detection	Records signing activity and identifies unusual transaction volume, destinations, times, or amounts.
Backup and recovery controls	Ensures keys can be recovered or replaced through controlled procedures without copying private material insecurely.

Response to an Actual Compromise

1. Immediately disable or revoke the compromised signing certificate/key according to the institution's incident-response procedure.
2. Preserve transaction logs, signature evidence, authentication records, and relevant system telemetry for investigation.
3. Determine the earliest reliable compromise time and identify all signatures created after that point.
4. Generate a new key pair using a secure key-generation process and bind it to the user's identity through the organization's certificate or key-management system.
5. Require stronger approval controls for high-value transactions and review whether any fraudulent transactions can be reversed or legally challenged.

Conclusion

Private-key compromise destroys the security assumption that the private key is under the exclusive control of the legitimate signer. Strong key protection, hardware-backed storage, MFA, transaction-level authorization, rapid revocation, and continuous monitoring are therefore essential in a financial signing environment. NIST FIPS 186-5 provides the current federal digital-signature standard and emphasizes the security properties that signatures are intended to provide.

3. Kerberos Authentication Failure in Distributed System

Case: A distributed system using Kerberos experiences frequent authentication failures because servers are not properly synchronized in time. The task is to analyze the effect of time synchronization and suggest solutions.

Introduction

Kerberos uses time-sensitive authenticators and tickets to reduce the risk of replaying old authentication messages. The Kerberos protocol specifies an allowable clock-skew window. If the client and application server clocks differ beyond that permitted skew, the server can reject the authenticator with KRB_AP_ERR_SKEW. RFC 4120 gives five minutes as an example of an allowable clock skew.



Figure 3: Simplified Kerberos authentication flow and the importance of synchronized clocks

How Clock Differences Cause Failure

- Kerberos tickets and authenticators contain timestamps that help the service determine whether an authentication request is fresh.
- If a client clock is ahead or behind the service by more than the configured tolerance, the service may treat the authenticator as invalid.
- The result is repeated login or service-access failures even when usernames, passwords, and tickets are otherwise correct.
- Large or unstable clock differences can also interfere with monitoring and incident investigation because logs from different hosts no longer align reliably.

Security Reason for the Time Requirement

The time check is a security mechanism, not merely an operational inconvenience. Kerberos uses freshness to reduce replay risk. RFC 4120 also describes replay-cache behavior for authenticators presented within the allowable clock-skew period. Therefore, simply increasing the allowed skew to eliminate errors is not the best primary solution because a larger window can weaken freshness controls.

Recommended Solutions

Solution	Implementation Guidance
Centralized NTP / secure time service	Synchronize clients, application servers, and KDC infrastructure to a reliable time source.
Monitor clock drift	Generate alerts before a host exceeds the Kerberos clock-skew threshold.
Harden time-service access	Restrict who can change system time and protect the network paths used for synchronization.
Redundant time sources	Use multiple trusted sources and define a controlled fallback strategy.
Validate the KDC first	The Key Distribution Center is critical infrastructure; verify its time and health before changing application settings.
Use a bounded skew policy	Keep the permitted skew aligned with organizational security requirements rather than using an excessively large tolerance.

Conclusion

Kerberos depends on synchronized time because timestamps contribute to authentication freshness and replay protection. The correct response is to establish reliable, monitored, and protected time synchronization across the distributed environment, while avoiding an unnecessarily large clock-skew allowance.

4. Certificate Authority Breach in PKI System

Case: A Certificate Authority is compromised and attackers' issue fake X.509 certificates for trusted domains. The task is to evaluate the impact on system trust and secure communication and propose detection and mitigation mechanisms.

Introduction

In a PKI, a trusted CA vouches for the binding between an identity and a public key through a certificate. A client validates the certificate chain back to a trusted root and checks relevant validity and revocation information. X.509 PKI therefore makes the CA a critical trust anchor. NIST notes that CA compromise can enable fraudulent certificates that may then be used to authenticate as another individual or system or to forge digital signatures.



Figure 4: X.509 trust chain and detection/response controls after a CA compromise

Impact on Trust and Secure Communication

- Fraudulent certificates may cause clients to trust an attacker-controlled public key as if it belonged to a legitimate domain.
- An attacker may be able to impersonate a trusted service in a TLS connection, enabling phishing, traffic interception, or unauthorized authentication depending on the affected certificate and system.
- The compromise can affect many relying parties because trust is inherited through the certificate chain rather than established separately for every server.
- Even strong cryptographic algorithms cannot compensate for a false trust binding. The problem is the authenticity of the certificate itself.
- Confidence in the CA's issued certificates may have to be reduced or removed, potentially causing widespread operational disruption.

Detection Mechanisms

- Monitor Certificate Transparency logs for unexpected certificates issued for organizational domains and investigate suspicious issuances.
- Use automated certificate inventory and expiry/issuance monitoring so newly issued certificates are compared with authorized requests.
- Check certificate revocation status using the applicable PKI mechanisms, including CRLs or OCSP where deployed.
- Review CA audit logs, certificate-request records, administrative access, and signing infrastructure for unauthorized activity.
- Use network and endpoint monitoring to identify unexpected certificate chains or anomalous TLS endpoints.

Mitigation and Recovery

1. Contain the compromised CA and stop unauthorized certificate issuance.
2. Identify the affected CA keys, certificates, subordinate CAs, and fraudulent certificates.
3. Revoke affected certificates and publish updated revocation information. RFC 5280 defines CRLs as signed, time-stamped lists of revoked certificates.
4. Replace compromised CA keys according to the organization's PKI recovery plan and reissue legitimate certificates.
5. Strengthen CA protection through offline/root-key controls, separation of duties, hardened signing infrastructure, restricted administrative access, and comprehensive audit logging.
6. Use certificate-issuance monitoring and Certificate Transparency where applicable to reduce the time between fraudulent issuance and detection.

Conclusion

A CA compromise attacks the trust foundation of PKI rather than a single endpoint. Effective recovery requires containment, identification of fraudulent certificates, revocation, controlled key replacement, and stronger issuance monitoring. NIST specifically addresses preparation and response to CA compromise and fraudulent certificate issuance, while RFC 5280 defines certificate and CRL mechanisms used in X.509 PKI.

5. Integrity Verification in Cloud Data Storage

Case: A cloud service provider uses hashing to verify file integrity, but users report tampering. Investigation finds improper hash-validation practices. The task is to analyze how incorrect hashing implementation causes integrity failures and recommend a robust mechanism.

Introduction

Integrity verification means obtaining assurance that stored or transmitted data has not been modified without authorization. NIST distinguishes integrity protection from simple data processing: an effective mechanism must allow the verifier to determine whether the data has changed since the integrity value was created.

How Incorrect Hash Validation Can Fail

- If the hash value is stored beside the file in the same attacker-controlled storage location, an attacker who changes the file may also replace the hash with a new hash of the modified file.
- If the application computes a hash but does not compare it against a trusted reference value, the calculation provides no decision about integrity.
- If the implementation uses an outdated or inappropriate hash algorithm, collision resistance or implementation assumptions may be weaker than required.
- If verification is performed only occasionally, a tampered file may remain undetected for a long period.
- If the comparison is implemented incorrectly, including unsafe parsing or non-constant-time comparisons in keyed verification contexts, the integrity check can become unreliable or expose additional attack opportunities.

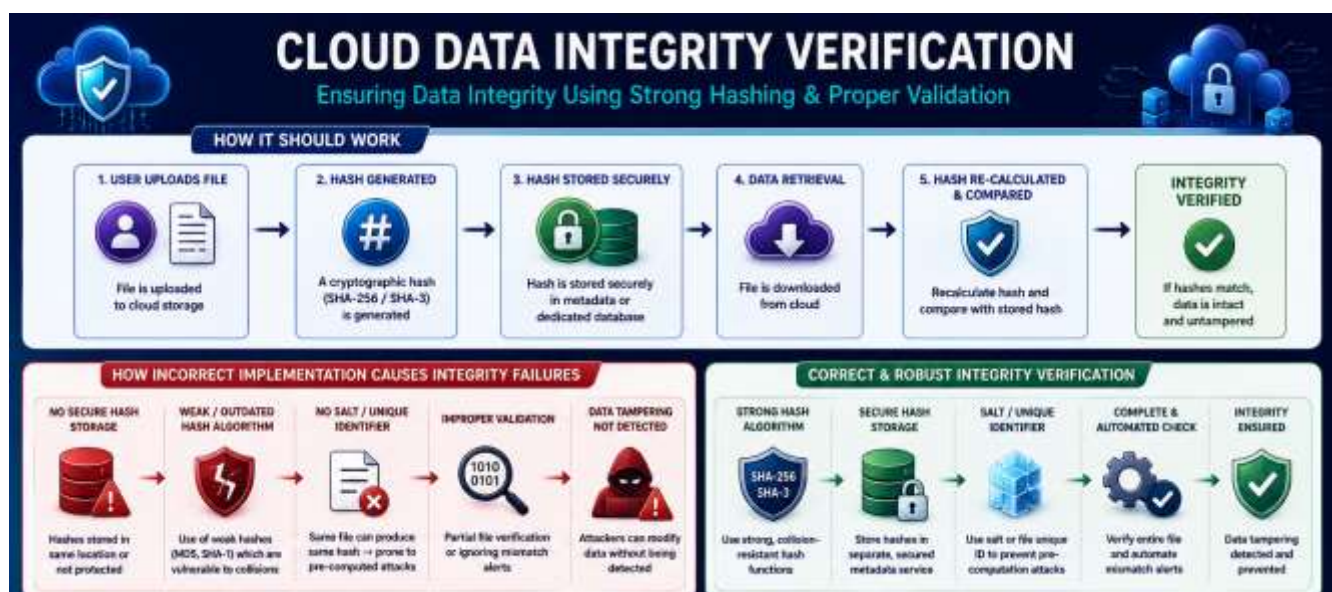


Figure 5: Robust cloud-file integrity verification using HMAC-SHA-256

Recommended Mechanism: Keyed Integrity Verification

For a cloud service that must detect unauthorized modification by parties who do not possess a verification secret, a keyed message authentication code such as HMAC is a strong design option. NIST defines HMAC as a mechanism that combines a cryptographic hash function with a shared secret key, and describes MACs as mechanisms for detecting intentional or accidental modifications.

- Generate a strong secret integrity key and keep it separate from the ordinary object/file storage plane.
- Compute HMAC-SHA-256 over the exact file bytes and store the resulting authentication tag in a protected metadata system.
- When a file is retrieved, recompute the HMAC using the trusted key and compare it with the stored tag.
- Reject the file or quarantine it when verification fails, and create an auditable security event.
- Use key rotation, access controls, backup, and secure key-management procedures so an attacker cannot simply replace both the file and its verification tag.
- For very high-assurance environments, consider digital signatures when non-repudiation or third-party verification is required; HMAC itself provides integrity and source authentication to parties sharing the secret, but not non-repudiation.

Operational Design

Layer	Recommended Control
File/object layer	Store immutable or versioned objects where possible and maintain access-control policies.
Integrity layer	Use HMAC-SHA-256 or another approved MAC when the verifier can securely hold a secret key.
Key-management layer	Keep keys outside the storage data path, restrict access, rotate keys, and audit key use.
Verification layer	Verify on upload, download, scheduled integrity scans, and after recovery/replication events as appropriate.
Monitoring layer	Alert on verification failures, repeated tampering attempts, unexpected metadata changes, and abnormal access.
Recovery layer	Retain known-good versions/backups so a detected tampered object can be restored safely.

Analysis and Justification

A plain hash can detect accidental changes only when the expected hash is itself trusted. If an attacker can change both the file and the stored hash, the verification process becomes circular. HMAC adds a secret key to the integrity computation, so an attacker who lacks the key cannot normally generate a valid tag for modified content. NIST describes HMAC as a keyed mechanism for message authentication

and integrity. This makes it a better fit for an internal cloud integrity service than an unprotected, standalone hash.

Conclusion

The cloud provider should not treat 'a hash exists' as proof of integrity. The expected integrity value must be protected from the same modification threat as the data. A keyed HMAC design, protected key management, repeated verification, logging, and recoverable versions provide a stronger and more complete integrity-verification architecture.